

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	D.HEMASREE	Reg. No.:	192524114
Section / Batch:	2025	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP Complete.	Q1

CODE:

```
import itertools
```

```
class SteinerTreeSolver:
```

```
    def __init__(self, num_vertices, edges, terminals):
        self.num_vertices = num_vertices
        self.edges = edges
        self.terminals = set(terminals)
        self.best_cost = float("inf")
        self.best_edges = []
```

```
    def find_connected_subgraph(self, selected_edges):
        parent = list(range(self.num_vertices))
```

```

def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x

def union(x, y):
    px, py = find(x), find(y)
    if px != py:
        parent[px] = py

total_cost = 0

for u, v, w in selected_edges:
    if find(u) != find(v):
        union(u, v)
        total_cost += w

roots = {find(t) for t in self.terminals}
return len(roots) == 1, total_cost

def solve(self):
    allowed_edges = list(self.edges)
    n = len(allowed_edges)

    for r in range(1, n + 1):
        for subset in itertools.combinations(allowed_edges, r):
            connected, cost = self.find_connected_subgraph(subset)

            if connected and cost < self.best_cost:
                self.best_cost = cost
                self.best_edges = list(subset)

    return self.best_cost, self.best_edges

def main():
    edges = [
        (0, 1, 3),
        (1, 2, 3),
        (2, 3, 3),
        (0, 2, 10),
        (1, 3, 20)
    ]

```

```
terminals = [0, 3]
```

```
solver = SteinerTreeSolver(4, edges, terminals)
cost, selected_edges = solver.solve()
```

```
print("Minimum Steiner Tree Cost:", cost)
print("Selected Edges:", selected_edges)
```

```
if __name__ == "__main__":
    main()
```

OUTPUT:

ASSESSMENT - C05-AT1-Constraint-Based Problem Solving

[← Back](#)Language: Python C C++ Java

QUESTIONS

Q1

Steiner Tree Problem (NP-Hard / Constraint Optimization Problem)
100 marks

1/1 answered

Q1 Steiner Tree Problem (NP-Hard / Constraint Optimization Problem)

100 marks

Given a graph with required terminal vertices, determine the minimum-cost tree connecting them. Clearly define constraints such as connectivity and inclusion of required nodes. Analyze how constraints influence the search space and solution complexity. Develop a backtracking-based solution to explore feasible trees. Apply pruning techniques to eliminate non-optimal trees. Justify how your solution ensures minimal cost and discuss NP-Hard complexity.

Your Code

```
1 import itertools
2 from collections import deque
3 class SteinerTreeSolver:
4     def __init__(self, num_vertices, edges, terminals):
5         self.num_vertices = num_vertices
6         self.edges = edges
7         self.terminals = sorted(terminals)
8         self.min_cost = float('inf')
9         self.best_tree_edges = []
10    def is_connected_with_edges(self, nodes, edges):
11        if not nodes:
12            return True
13        adj = {i: [] for i in nodes}
14        node_set = set(nodes)
15        for u, v, w in edges:
16            if u in node_set and v in node_set:
17                adj[u].append(v)
18                adj[v].append(u)
19        start = nodes[0]
20        visited = set([start])
21        q = deque([start])
22        while q:
```

Input

stdin input

Output

```
Minimum Steiner Tree Cost: 10
Edges in one optimal Steiner tree:
(0, 3, 10)
```

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
-------------------	--------------------	--------------------

Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____